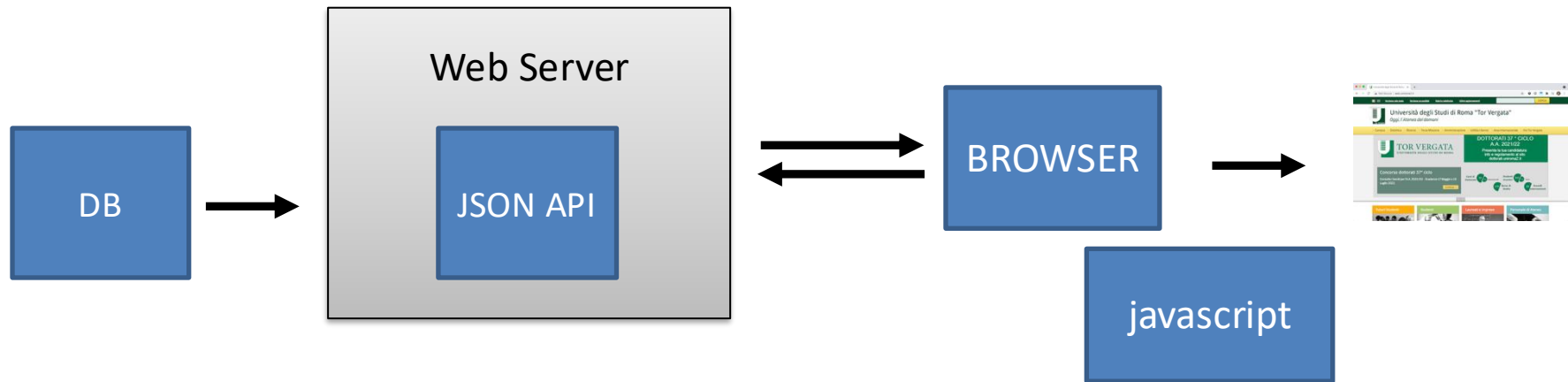


Api JSON

Architettura



Cos'è una REST API?

- Un'API è codice che permette a due software di comunicare tra loro
- REST è uno stile architetturale per costruire API web usando HTTP
- È lo standard de facto: preferito per leggerezza, semplicità e compatibilità con qualsiasi linguaggio
- **Usata da:** Google Maps, Spotify, Netflix, Amazon, Twitter, Uber

I 3 elementi e la struttura della request

- **3 elementi:** Client (chi fa la richiesta) · Server (chi controlla la risorsa) · Resource (il dato: JSON, testo, immagine)
- **Una request contiene:** HTTP method · Endpoint (URL) · Header · Body
- **HTTP method:** POST (create) · GET (read) · PUT (update) · DELETE (delete)
- **Endpoint:** l'URL che identifica la risorsa nel sistema

REST vs Web Classico

- Nel mondo WEB viene utilizzato il metodo GET per eseguire qualsiasi tipo di interazione con il server.

```
GET /addCustomer?name=Rossi
```

GET deve solo **recuperare** dati, non modificarli!!

- Per creare una risorsa in REST uso la POST

```
POST /customers
```

```
Body: { "name": "Rossi" }
```

Web Operations

products

users

orders

http://my-url/**addNewProduct**

/getProduct

/updateProduct

/deleteProduct

/getProductbyOrder

/getOrderByUser

REST Operations

/addNewProduct

/getProduct

/updateProduct

/deleteProduct

/getProductbyOrder

/getOrderbyUser

POST /products

GET /products/3

PUT /products/3

PATCH /products/3

DELETE /products/3

GET /orders/4/products

GET /users/9/orders

Operazioni CRUD

- Acronimo per:
create, read (aka retrieve), update, and delete
- Operazioni di base che posso fare su una risorsa
 - **Create** (creare una risorsa)
 - **Read** o **Retrieve** (leggere una risorsa)
 - **Update** (aggiornare una risorsa)
 - **Delete** (eliminare una risorsa)

REST e CRUD

Metodo HTTP	Operazione CRUD	Descrizione
POST	Create	Crea una nuova risorsa
GET	Read	Ottiene una risorsa esistente
PUT	Update	Aggiorna una risorsa o ne modifica lo stato
DELETE	Delete	Elimina una risorsa

Esempio

Risorsa	GET read	POST create	PUT update	DELETE
<i>/books</i>	Ritorna una lista di libri	Crea un nuovo libro	Aggiorna i dati di tutti i libri	Elimina tutti i libri
<i>/books/145</i>	Ritorna uno specifico libro	metodo non consentito (405)	Aggiorna uno specifico libro	Elimina uno specifico libro

Entry Point

- Unico punto di ingresso API (es. /api/v1/)
- Fornisce:
 - Info versioni
 - Liste di collezioni
 - Risorse principali



Evitare strutture URL ambigue o opache

Struttura delle URL

URL	Description
/api	The API entry point
/api/:coll	A top-level collection named "coll"
/api/:coll/:id	The resource "id" inside collection "coll"
/api/:coll/:id/:subcoll	Sub-collection "subcoll" under resource "id"
/api/:coll/:id/:subcoll/:subid	The resource "subid" inside "subcoll"

```

/endpoint
  /collection1
    /resource1
    /resource2
    /resource3
  /collection2
    /resource1
    /resource2

```

...

REST e Status Code

- 200 – OK – Tutto bene
- 201 – OK – E' stata creata una nuova risorsa
- 204 – OK – La risorsa è stata cancellata con successo
- 304 – Not modified – I dati non sono cambiati. Il cliente può utilizzare i dati nella cache
- 400 – Bad Request – Richiesta non valida. L'errore esatto dovrebbe essere spiegato nel payload dell' errore (di cui ne parleremo a breve). Per esempio. "Il JSON non è valido"
- 401 – Unauthorized – La richiesta richiede una autenticazione dell'utente
- 403 – Forbidden – Il server ha capito la richiesta, ma in base ai diritti del richiedente l'accesso non è consentito.
- 404 – Not Found – Non vi è alcuna risorsa dietro l'URI richiesto.
- 422 – Unprocessable Entity – deve essere usato se il server non può elaborare il entity, ad esempio se un'immagine non può essere formattata o campi obbligatori sono mancanti nel payload.
- 500 – Internal Server Error – gli sviluppatori di API dovrebbero evitare questo errore. Se si verifica un errore globale dell'applicazione, lo stacktrace deve essere loggato e non inviato nella risposta all'utente.

Negoziare il formato: Accept e Content-Type

- Client e server si accordano sul formato dei dati tramite header HTTP
- **Accept** (Client → Server): il client indica i formati che accetta
 - es. Accept: application/json
- **Content-Type** (Server → Client): il server indica il formato della risposta
 - es. Content-Type: application/json
- **Formati comuni:** application/json · application/xml · text/plain

JSON formatting

```
{  
  "id": 1,  
  "name": "cerulean",  
  "year": 2000,  
  "color": "#98B2D1",  
  "pantone_value": "15-4020"  
}
```

JSON formatting

JSEND

<https://github.com/omniti-labs/jsend>

```
{
  "id": 1,
  "name": "cerulean",
  "year": 2000,
  "color": "#98B2D1",
  "pantone_value": "15-4020"
}
```



```
{
  "status": "success",
  "data": {
    "id": 1,
    "name": "cerulean",
    "year": 2000,
    "color": "#98B2D1",
    "pantone_value": "15-4020"
  }
}
```

EXPRESS REST API

GET

```
app.get('/api/v1/products', (req, res) => {  
  res.status(200).json({  
    status: 'success',  
    data: {  
      products: products,  
    },  
  });  
});
```

GET

```
app.get('/api/v1/products/:id', (req, res) => {
  console.log(req.params);

  const prod = products.find((el) => el.id == req.params.id);
  console.log(prod);
  if (prod == undefined) {
    res.status(404).json({
      status: 'fail',
      message: 'ID non trovato',
    });
  } else {
    res.status(200).json({
      status: 'success',
      data: {
        product: prod,
      },
    });
  }
});
```

POST

```
app.post('/api/v1/products', (req, res) => {
  const newId = products[products.length - 1].id + 1;
  const newProd = Object.assign({ id: newId }, req.body);

  products.push(newProd);
  res.status(201).json({
    status: 'success',
    data: { product: newProd },
  });
});
```

PUT/PATCH

```
app.patch('/api/v1/products/:id', (req, res) => {
  const prod = products.find((el) => el.id == req.params.id);
  if (prod == undefined) {
    res.status(404).json({
      status: 'fail',
      message: 'ID non trovato',
    });
  } else {
    // Update ....

    res.status(200).json({
      status: 'success',
      data: {
        product: prod,
      },
    });
  }
});
```

DELETE

```
app.delete('/api/v1/products/:id', (req, res) => {
  const prod = products.find((el) => el.id == req.params.id);
  if (prod == undefined) {
    res.status(404).json({
      status: 'fail',
      message: 'ID non trovato',
    });
  } else {
    // Delete ....
    res.status(204).json({
      status: 'success',
      data: null,
    });
  }
});
```